

absa-cloud

Architettura dell'ambiente di sviluppo locale

Piattaforma 3-tier a microservizi per l'Aspect-Based Sentiment Analysis di recensioni bancarie — inferenza asincrona e scalabile.



Disaccoppiamento asincrono

- 01** L'utente invia una recensione: l'**API** la salva su PostgreSQL come pending e pubblica **un messaggio** nella coda.
- 02** L'API risponde **202 Accepted** immediatamente — non attende l'inferenza, l'interfaccia resta reattiva.
- 03** I **worker** consumano dalla coda `absa.reviews` al proprio ritmo ed eseguono l'analisi ABSA.
- 04** I risultati tornano su PostgreSQL. **Scalare i worker = scalare la capacità di inferenza**, senza toccare il resto.

Frontend / Streamlit

- Interfaccia utente del sistema: invio recensioni e dashboard del sentiment.
- Due schede: **Invia recensione** (analisi) e **Dashboard** (aggregazione per aspetto).
- Comunica con l'API via **HTTP / REST**; nessun accesso diretto a coda o database.

TIER 1 · PRESENTATION

Frontend

Streamlit

PORTA	:8501
PROTO	HTTP
DEPENDS	api

API / FastAPI

- Riceve le recensioni su **POST /reviews** e le salva su PostgreSQL come pending.
- Pubblica **un messaggio per recensione** nella coda — unità ritentabile e metrica di scaling.
- Risponde subito **202 Accepted**: non blocca il client in attesa dell'inferenza.
- Swagger su /docs.

TIER 2 · APPLICATION

API

FastAPI

PORTA	:8000
RISPOSTA	202
PUBLISH	AMQP

RabbitMQ / coda asincrona

- Coda `absa.reviews`: assorbe i picchi di traffico e protegge i worker.
- La **lunghezza della coda** è il segnale che pilota l'autoscaling (KEDA).
- Management UI per ispezionare la coda su `:15672`.

BROKER · CODA

RabbitMQ

queue: `absa.reviews`

AMQP :5672

MGMT :15672

PATTERN 1 msg / recensione

Worker / PyABSA · LCF-ATEPC

- Consuma da `absa.reviews` ed esegue l'inferenza → coppie (**aspetto, sentiment**).
- **Micro-batching**: accumula N messaggi e chiama `infer_batch()` — BERT generalmente su CPU è molto più efficiente in batch.
- Modello LCF-ATEPC (fine-tuned in un altro progetto); modalità mock di default per test rapidi.
- **Scalabile orizzontalmente** — è il componente che si replica per aumentare la capacità.

TIER 2 · APPLICATION

Worker

PyABSA · LCF-ATEPC

CONSUMA	AMQP
REPLICAS	0 → 5
OUTPUT	SQL

PostgreSQL / persistenza

- Unico strato di persistenza: **recensioni e risultati ABSA**.
- Stato della recensione: pending → processed dopo l'inferenza.
- Dati su **volume persistente** (PVC 2Gi, ReadWriteOnce).
- Schema inizializzato da `db/init.sql`.

TIER 3 · DATA

PostgreSQL

16-alpine

PORTA	:5432
STORAGE	PVC 2Gi
ACCESS	RWO

KEDA / scaling pilotato dalla coda



- Lo ScaledObject worker-keda-scaler osserva la lunghezza della coda RabbitMQ.
- **+1 pod ogni 30 messaggi** in coda; nessun intervento manuale.
- **Scale-to-zero:** con la coda vuota i worker scendono a 0 — risorse a riposo.

TRIGGER · RABBITMQ	
METRICA	QueueLength
VALUE	30 msg / pod
MIN	0
MAX	5
POLLING	10s
COOLDOWN	30s